

Algorithmen und Datenstrukturen



Ziele der heutigen Lerneinheit

Sie verstehen...

- wie kantengewichtete Graphen formal definiert sind
- die Kernprinzipien der Graphen-Relaxation
- die grundlegenden Unterschiede zwischen gieriger Suche (Dijkstra) und systematischem Durchlaufen (Bellman-Ford).
- das Konzept einer informierten Suche

Sie können...

- den Dijkstra- und Bellman-Ford-Algorithmus schrittweise auf einem gegebenen, bewerteten Graphen ausführen
- bestehende Graphen auf negative Zyklen prüfen
- die geschätzten Gesamtkosten für den A*-Algorithmus ermitteln und den Ablauf bis zum Zielknoten darstellen

Bewertete Graphen

Bewertete Graphen

Definition

Bisher haben wir eine Kante als „einen Schritt“ im Graphen betrachtet, um die Länge von Pfaden zu ermitteln.

In einer allgemeineren Sichtweise können die Kanten aber unterschiedliche Längen haben, die z.B. zu unterschiedlichen Kosten beim Beschreiten der Kante führen können.

Ja sogar „negative Kosten“ sind denkbar, wenn beim Beschreiten eine Auszahlung erfolgt.

Graphen $G = (V, E)$, die mit Hilfe einer Bewertungsfunktion $w: E \rightarrow \mathbb{R}$ reellwertige Kantengewichte besitzen, heißen ***bewertete Graphen***.

Bewertete Graphen

Gewicht eines Pfades

Das Gewicht w eines Pfades

$$p = \langle v_0, v_1, \dots, v_k \rangle$$

ist die Summe der Gewichte aller Kanten,
aus denen er zusammengesetzt ist:

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

Bewertete Graphen

Kürzester Pfad

Sei P die Menge aller Pfade p von u nach v :

$$P = \{p: u \xrightarrow{p} v\}$$

Das minimale Gewicht eines Pfades von u nach v ist definiert durch:

$$\delta(u, v) = \begin{cases} \min_{p \in P} w(p), & P \neq \emptyset \\ \infty, & \text{sonst} \end{cases}$$

Ein kürzester Pfad $\hat{p}$ von u nach v besitzt dann das Gewicht

$$w(\hat{p}) = \delta(u, v)$$

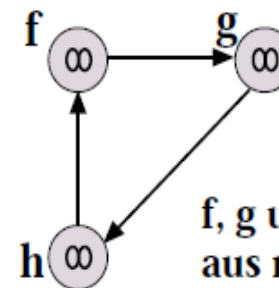
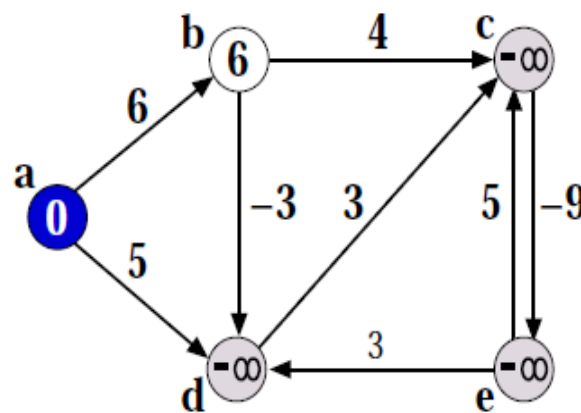
Suche nach kürzesten Pfaden in bewerteten Graphen

Suche nach kürzesten Pfaden

Problem: Negativ gewichtete Zyklen

Ist von einem Knoten s aus ein negativ gewichteter Zyklus erreichbar, dann sind Gewichte des kürzesten Pfades nicht wohl definiert, da mit jedem Durchlaufen des Zyklus der Pfad immer kürzer wird, so dass

$$\delta(s, v) = -\infty$$



f, g und h sind von Knoten a aus nicht erreichbar

Suche nach kürzesten Pfaden

Obere Schranke für das Gewicht des kürzesten Pfades

In den folgenden Algorithmen merken wir uns für jeden Knoten v eine obere Schranke d für das Gewicht des kürzesten Pfades vom Startknoten s zu v .

Zu Beginn gilt:

$$d(v) = \begin{cases} 0, & v = s \\ \infty, & v \neq s \end{cases}$$

Suche nach kürzesten Pfaden

Relaxation

Während der Ausführung der Algorithmen wird die obere Schranke als Schätzung des kürzesten Pfades schrittweise reduziert.

Dabei wird beim Besuch des Knotens u getestet, ob – sofern eine Kante (u, v) existiert – der bislang gültige Schätzwert für v aktualisiert werden muss. Diesen Schritt nennt man **Relaxieren**.

Suche nach kürzesten Pfaden

Relaxation

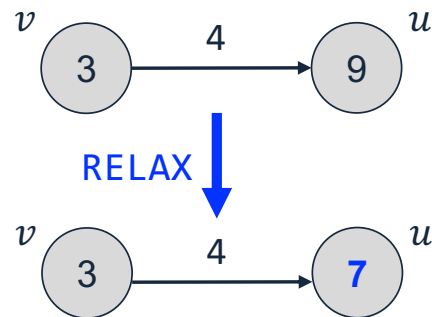
RELAX(e)

```

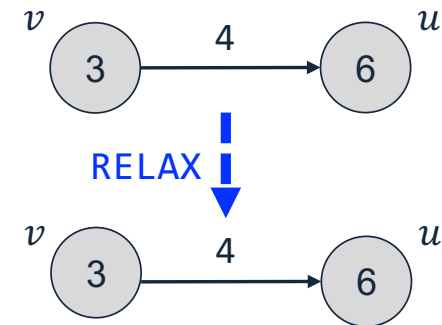
1:  $(v, u) \leftarrow e$ 
2: if  $v.dist + w(e) < u.dist$  then
3:    $u.dist \leftarrow v.dist + w(e)$ 
4:    $u.pred \leftarrow v$ 
5: end if
  
```

 $\triangleright e \in E \subseteq V \times V$

Kürzen des Pfades:



Kein Kürzen des Pfades:



Dijkstra Algorithmus

Dijkstra Algorithmus

Wer war Dijkstra?



Edsger W. Dijkstra
(1930 – 2002)

Niederländischer Informatiker

- 1960 Compiler für Algol 60
- 1965 Semaphore zur Synchronisation
- 1968 „Go To Considered Harmful“
- 1971 „Dining Philosophers“
- 1972 Turing Award
- 1979 „Separation of Concerns“

Dijkstra Algorithmus

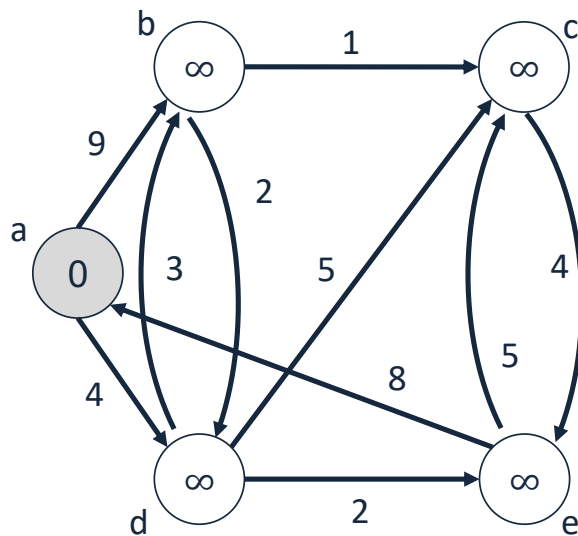
Grundidee

Ähnlich wie bei der Breitensuche werden die weiter zu untersuchenden Knoten in einer Warteschlange *queue* vorgehalten. Allerdings wird nicht das FIFO-Prinzip verwendet, sondern stets der Knoten mit dem kleinsten d -Wert ausgewählt (Vorrangwarteschlange).

Voraussetzung

Kantengewichte sind nicht negativ: $w: E \rightarrow \mathbb{R}_0^+$

Dijkstra Algorithmus



queue:

a
0

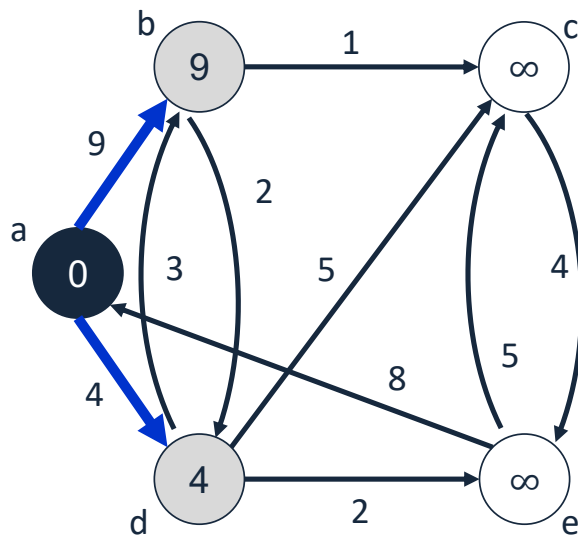
Start:

Alle Distanzen werden auf unendlich gesetzt.

Lediglich der Startknoten erhält die Distanz 0.

Daher ist er der Knoten mit der geringsten Distanz und wird als nächstes gewählt.

Dijkstra Algorithmus



queue:

b	d
9	4

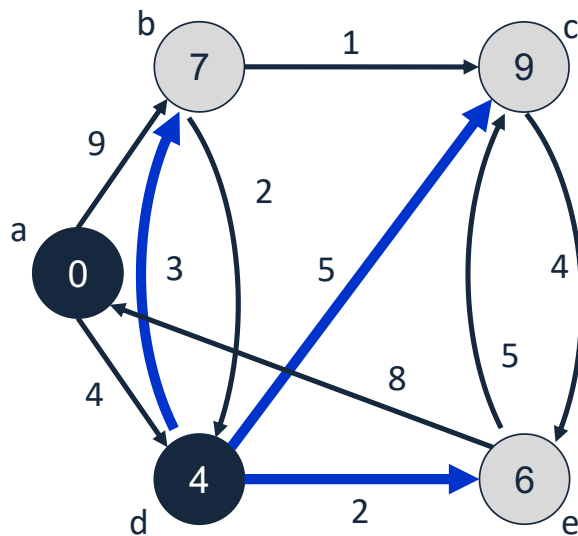
Nächster Schritt:

Entlang aller ausgehenden Kanten von a wird relaxiert.

Der Knoten a wird aus *queue* entfernt und ist abschließend berechnet.

Der Knoten d hat jetzt die geringste Distanz und wird gewählt.

Dijkstra Algorithmus



queue:

b	c	e
7	9	6

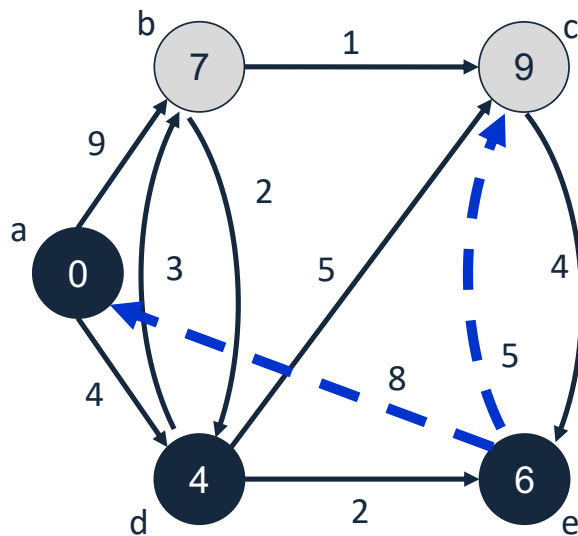
Nächster Schritt:

Entlang aller ausgehenden Kanten von d wird relaxiert.

Der Knoten d wird aus *queue* entfernt und ist abschließend berechnet.

Der Knoten e hat jetzt die geringste Distanz und wird gewählt.

Dijkstra Algorithmus



queue:

b	c
7	9

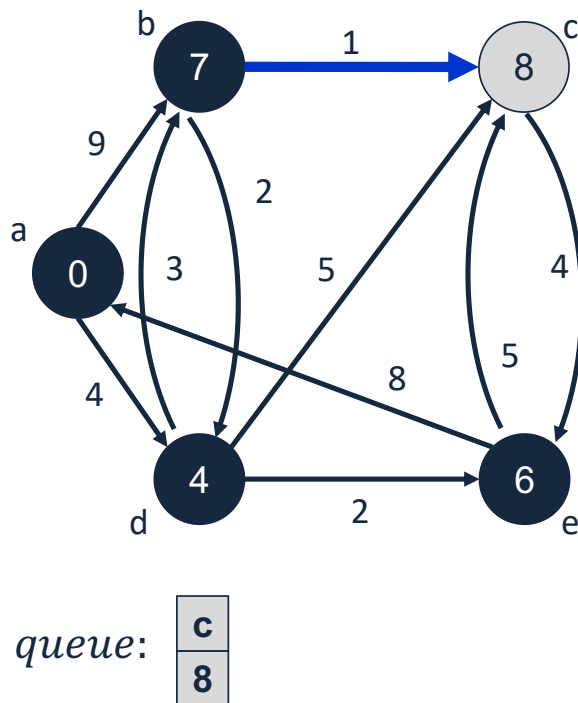
Nächster Schritt:

Die ausgehenden Kanten von *e* bringen keine Verbesserungen.

Der Knoten *e* wird aus *queue* entfernt und ist abschließend berechnet.

Der Knoten *b* hat jetzt die geringste Distanz und wird gewählt.

Dijkstra Algorithmus



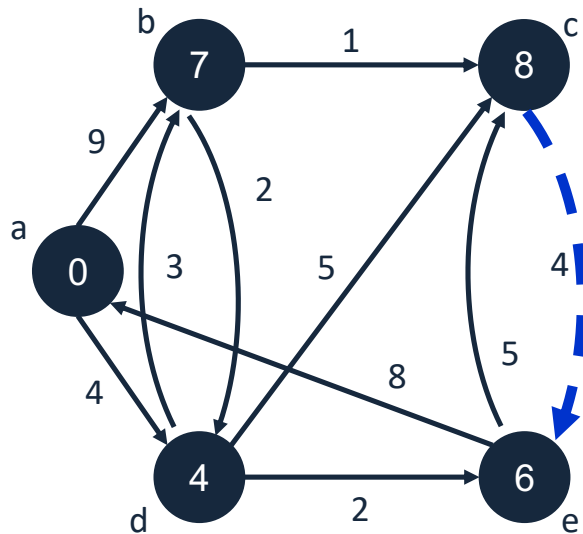
Nächster Schritt:

Entlang aller ausgehenden Kanten von *b* wird relaxiert.

Der Knoten *b* wird aus *queue* entfernt und ist abschließend berechnet.

Der Knoten *c* hat jetzt die geringste Distanz und wird gewählt.

Dijkstra Algorithmus



queue: $\emptyset$

Nächster Schritt:

Die ausgehenden Kanten von *c* bringen keine Verbesserungen.

Der Knoten *c* wird aus *queue* entfernt und ist abschließend berechnet.

Die *queue* ist nun leer, der Algorithmus terminiert.

Dijkstra Algorithmus

DIJKSTRA(V, E, s)

```
1: queue  $\leftarrow$  new PriorityQueue            $\triangleright$  Vorrangwarteschlange
2: for each  $v \in V$  do
3:    $v.dist \leftarrow \infty$ 
4:    $v.pred \leftarrow \mathbf{NIL}$ 
5:   queue.insert( $v$ )
6: end for
7:  $s.dist \leftarrow 0$ 
8: while queue not empty do
9:    $v \leftarrow queue.extract\_min()$ 
10:  for each  $e : v \rightarrow u \in E$  do     $\triangleright$  Alle von  $v$  ausgehenden Kanten
11:    RELAX( $e$ )
12:  end for
13: end while
```

Dijkstra Algorithmus

Analyse

Jede Kante wird nur einmal betrachtet.

Jeder Knoten wird ebenfalls nur einmal betrachtet, allerdings muss jedesmal der nächste Knoten aus den verbleibenden ausgewählt werden. Bei einer Implementierung als lineare Suche in einem Array ergibt sich

$$O(|V|^2 + |E|)$$

Verwendet man für die Realisierung der Vorrangwarteschlange einen Min-Heap (vgl. Max-Heap beim Heap-Sort), so kann

$$O(|V| \log |V| + |E|)$$

erreicht werden.

Bellman-Ford Algorithmus

Bellman-Ford Algorithmus

Grundidee

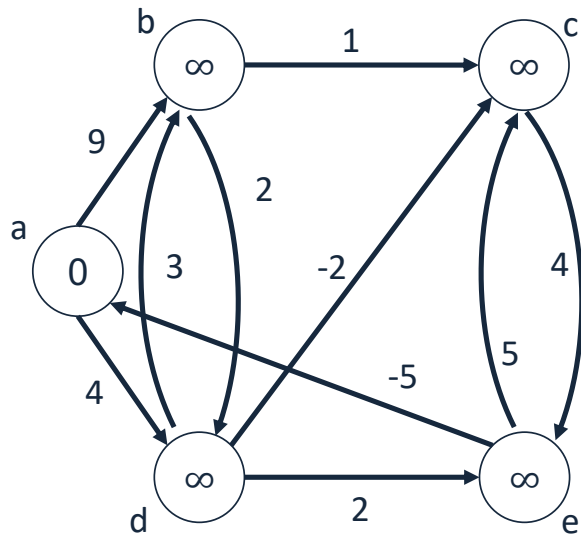
Der kürzeste Pfad ist bei $|V|$ Knoten maximal $|V| - 1$ Kanten lang. Daher werden nach einer Initialisierung analog zum Dijkstra-Algorithmus $|V| - 1$ mal alle Kanten durchlaufen und relaxiert. Ausgehend vom Startknoten sind damit alle kürzesten Pfade gesetzt.

Gibt es dann noch immer eine verbessernde Relaxation, so muss ein Zyklus mit negativem Kantengewicht enthalten sein.

somit:

Kantengewichte dürfen auch negativ sein. Das Auftreten eines Zyklus mit negativem Gewicht wird erkannt.

Bellman-Ford Algorithmus

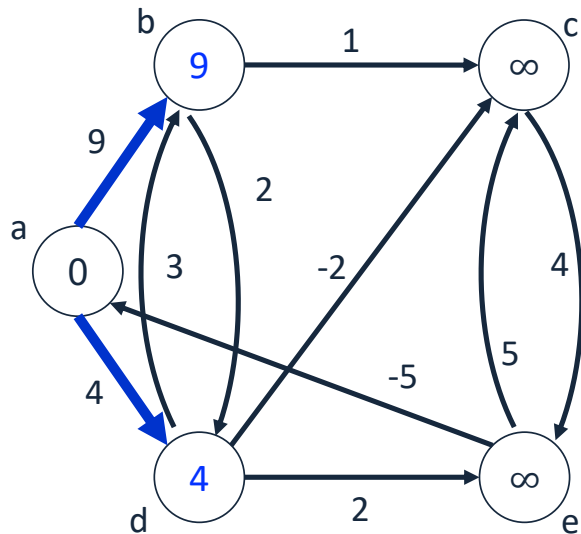


Start:

Alle Distanzen werden auf unendlich gesetzt.

Lediglich der Startknoten erhält die Distanz 0.

Bellman-Ford Algorithmus

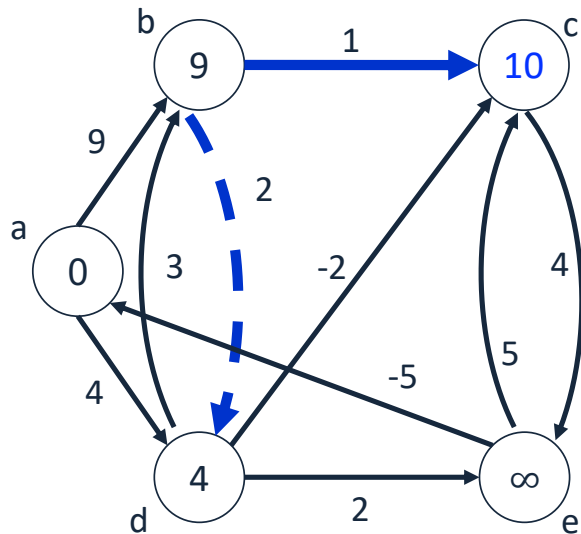


Nächster Schritt

Nur bei Knoten mit einer Distanz ungleich ∞ kann sinnvoll eine Relaxation durch geführt werden.

1. Durchlauf bei 5 Knoten

Bellman-Ford Algorithmus

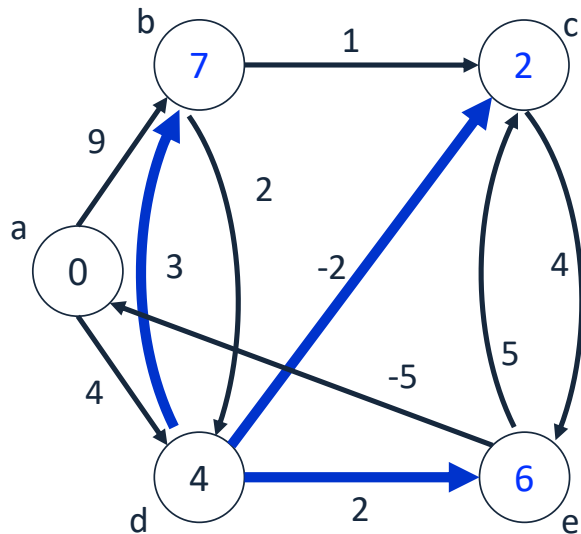


Nächster Schritt

Nur bei Knoten mit einer Distanz ungleich ∞ kann sinnvoll eine Relaxation durchgeführt werden.

2. Durchlauf bei 5 Knoten

Bellman-Ford Algorithmus

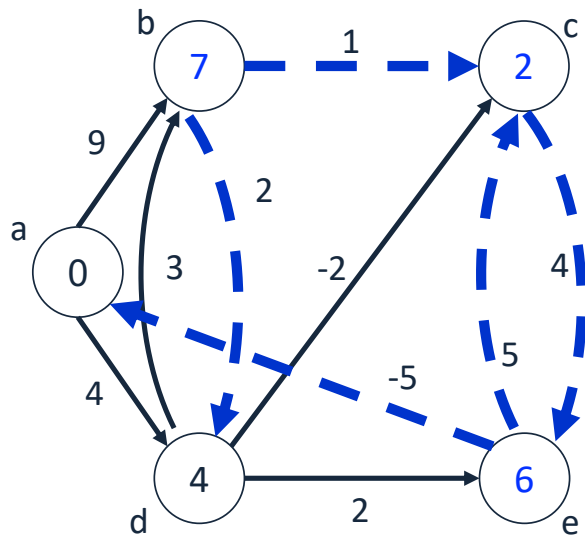


Nächster Schritt

Nur bei Knoten mit einer Distanz ungleich ∞ kann sinnvoll eine Relaxation durch geführt werden.

2. Durchlauf bei 5 Knoten

Bellman-Ford Algorithmus



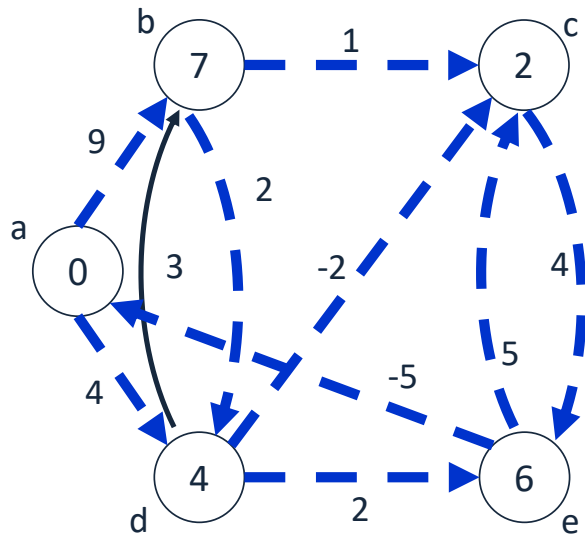
Nächster Schritt

Nur bei Knoten mit einer Distanz ungleich ∞ kann sinnvoll eine Relaxation durchgeführt werden.

3. Durchlauf bei 5 Knoten

Keine Verbesserung mehr möglich.

Bellman-Ford Algorithmus



Nächster Schritt

Auch beim 4. und 5. Durchlauf kann keine Verbesserung mehr erreicht werden.

Daher sind keine Zyklen mit negativem Gewicht enthalten.

Bellman-Ford Algorithmus

Analyse

Der Bellman-Ford-Algorithmus besitzt eine Laufzeit von

$$O(|V| \cdot |E|)$$

BELLMAN-FORD(V, E, s)

```

1: for each  $v \in V$  do
2:    $v.dist \leftarrow \infty$ 
3:    $v.pred \leftarrow \text{NIL}$ 
4: end for
5:  $s.dist \leftarrow 0$ 
6: for  $i \leftarrow 1$  to  $|V| - 1$  do
7:   for each  $e \in E$  do                                ▷ Alle Kanten
8:     RELAX( $e$ )
9:   end for
10: end for
11: for each  $e : v \rightarrow u \in E$  do                        ▷ Alle Kanten
12:   if  $v.dist + w(e) < u.dist$  then
13:     return False
14:   end if
15: end for
16: return True

```

A* Algorithmus

A* Algorithmus

Informierte Suche

Der **A* Algorithmus** findet den kürzesten Weg von einem Startknoten s zu einem Zielknoten z .

Wie der Dijkstra Algorithmus erfordert A* ausschließlich positive Kantengewichte.

Er benötigt eine **Heuristik**

$$h: V \rightarrow \mathbb{R}_0^+$$

die vorab die Kosten für das Erreichen des Zielknotens abschätzt.

Damit handelt es sich um eine **informierte Suche**.

A* Algorithmus

Heuristik für den A* Algorithmus

Die Heuristik muss **monoton** sein, d.h.

- der Schätzwert der Heuristik darf die wahren Kosten für keinen Knoten übersteigen.
- der Schätzwert für einen Vorgänger darf die geschätzten Kosten für einen Nachfolger im Pfad nicht überschreiten:

$$h(v) \leq w(v, u) + h(u)$$

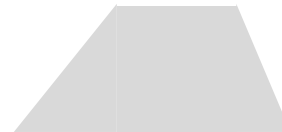
Diese Eigenschaften sind z.B. durch die Verwendung der „Luftlinien“-Entfernung zum Zielknoten gegeben.

A* Algorithmus

Geschätzte Gesamtkosten

Mit Hilfe der Heuristik und den sukzessive ermittelten Kosten für das Erreichen eines Knotens lassen sich stets die Gesamtkosten f vom Startknoten s bis zum Zielknoten z abschätzen:

$$f(v) = d(v) + h(v)$$



Kosten von s zu v



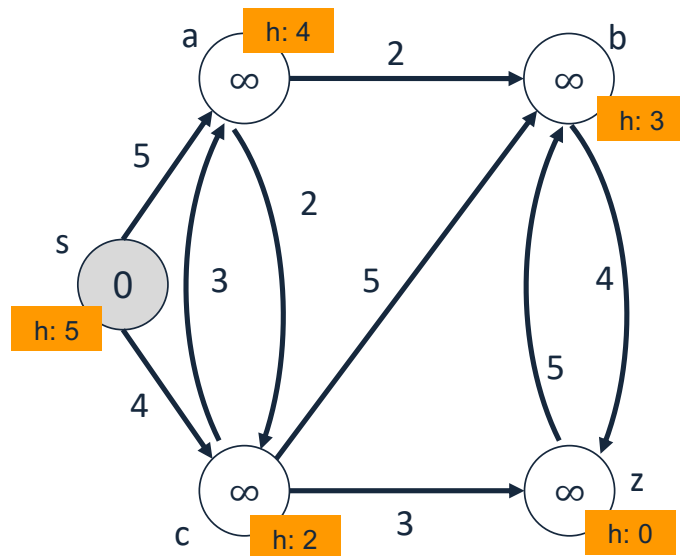
Kosten von v zu z

A* Algorithmus

Grundidee

Der A* Algorithmus ähnelt dem Verfahren von Dijkstra, allerdings wird nicht der Knoten für die weitere Expansion ausgewählt, der mit den geringsten Kosten d erreicht werden kann, sondern der Knoten, der die geringsten geschätzten Gesamtkosten f ausweist.

A* Algorithmus



queue:

s
d 0
h 5
f = d + h 5

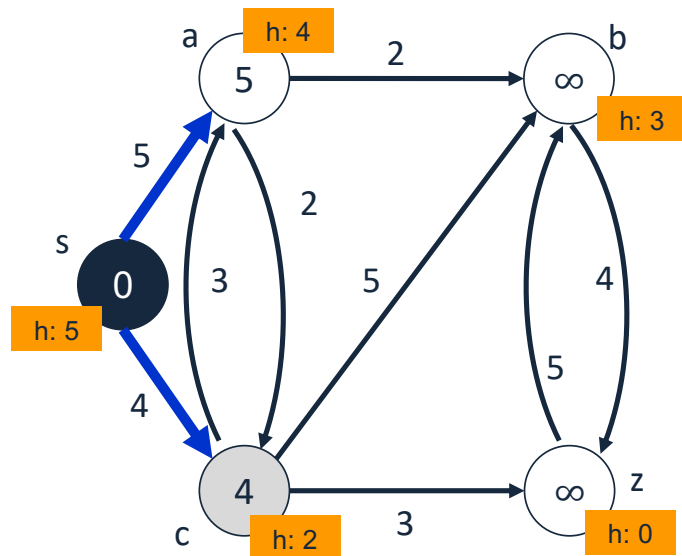
Start:

Alle Distanzen werden auf unendlich gesetzt.

Lediglich der Startknoten erhält die Distanz 0 und wird in die *queue* eingesetzt.

Dort ist er der Knoten mit den geringsten geschätzten Gesamtkosten f und wird daher als nächstes gewählt.

A* Algorithmus



queue:

	a	c
d	5	4
h	4	2
f = d + h	9	6

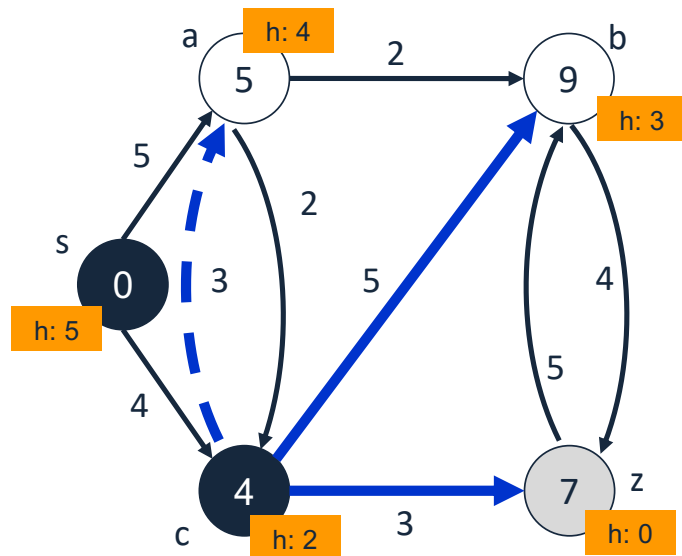
Nächster Schritt:

Entlang aller ausgehenden Kanten von *s* wird relaxiert und die erreichten Knoten in *queue* aufgenommen.

Der Knoten *s* wird aus *queue* entfernt und ist abschließend berechnet.

Der Knoten *c* hat jetzt die geringsten Gesamtkosten und wird expandiert.

A* Algorithmus



queue:

	a	b	z
d	5	9	7
h	4	3	0
f = d + h	9	12	7

Nächster Schritt:

Entlang aller ausgehenden Kanten von *c* wird relaxiert und die erreichten Knoten in *queue* aufgenommen.

Der Knoten *c* wird aus *queue* entfernt und ist abschließend berechnet.

Der Knoten *z* hat jetzt die geringsten Gesamtkosten. Es ist der Zielknoten und der Algorithmus terminiert.

A* Algorithmus

Korrektheitsüberlegung

Ist für den gewählten Knoten v bereits der kürzeste Pfad gefunden?

Gäbe es einen alternativen kürzeren Weg über u

$$d(u) + w(u, v) < d(v)$$

dann

$$f(u) < f(v)$$

und v wäre nicht gewählt worden.

Begründung:

$$d(u) + w(u, v) < d(v)$$

$$d(u) + h(u) + w(u, v) < d(v) + w(u, v) + h(v)$$

$$d(u) + h(u) < d(v) + h(v)$$

Aufgrund der Monotonie
 $h(u) \leq w(u, v) + h(v)$

A* Algorithmus

COST(v)

```

1:  $\triangleright$  Die Kosten eines Pfades vom Start zum Ziel über  $v$   $\triangleleft$ 
2: return  $v.dist + heuristic(v)$ 

```

A*(V, E, s, z)

```

1:  $queue \leftarrow$  new PriorityQueue  $\triangleright$  Vorrangwarteschlange
2: for each  $v \in V$  do
3:    $v.dist \leftarrow \infty$ 
4:    $v.pred \leftarrow \text{NIL}$ 
5:    $v.color \leftarrow \text{WHITE}$ 
6: end for
7:  $s.dist \leftarrow 0$ 
8:  $s.color \leftarrow \text{GRAY}$ 
9:  $queue.insert(s)$ 
10: while  $queue$  not empty do
11:    $v \leftarrow queue.extract\_min()$   $\triangleright$  Basierend auf COST
12:   if  $v = z$  then
13:     return True
14:   end if
15:   for each  $e : v \rightarrow u \in E$  do  $\triangleright$  Alle von  $v$  ausgehenden Kanten
16:     if  $u.color = \text{BLACK}$  then
17:       continue
18:     end if
19:      $cost \leftarrow v.dist + w(e) + heuristic(u)$ 
20:     if  $u.color = \text{GRAY} \wedge cost > \text{COST}(u)$  then
21:       continue
22:     end if
23:      $u.pred \leftarrow v$ 
24:      $u.dist \leftarrow v.dist + w(e)$ 
25:      $u.color \leftarrow \text{GRAY}$ 
26:      $queue.insert\_or\_update(u)$ 
27:   end for
28:    $v.color \leftarrow \text{BLACK}$ 
29: end while
30: return False

```

A* Algorithmus

Analyse

Die Zeitkomplexität ist abhängig von der Implementierung der Auswahl des Knotens mit minimalem f . Wird hier ein Min-Heap gewählt, dann beträgt die Laufzeit im Worst-Case

$$O(|V|^2)$$

Diese schlechte Zeitkomplexität trifft jedoch im durchschnittlichen Fall nicht zu, da die Stärke des A*-Algorithmus seine zielgerichtete Suche ist und im Vergleich zur Gesamtzahl der Knoten meist nur ein geringer Teil davon untersucht wird.

Vielen Dank für die Aufmerksamkeit!